WEEK 8 · LESSON 2

# FastAPI Hands-On Tutorial

Build your first API from scratch — one concept at a time

Mohamed bin Zayed University of Artificial Intelligence

# Your First API

A complete web API in 5 lines of Python

main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def hello():
    return {"message": "Hello, World!"}
```

```
$ uvicorn main:app --reload
```

## What each line does

| | | |
|---|---|---|
| 1 | **Line 1** | Import the framework |
| 2 | **Line 3** | Create your app instance |
| 3 | **Line 5** | Declare a GET endpoint at / |
| 4 | **Line 6-7** | Return JSON automatically |

**Visit localhost:8000**

```
{"message": "Hello, World!"}
```

**Visit localhost:8000/docs**

Interactive Swagger UI — for free!

# ASGI — The Protocol

How Python web apps talk to web servers

## WSGI (old standard)

- Synchronous only — one request at a time per worker
- No native WebSocket support
- Used by Flask, Django (traditional)
- Good enough for simple CRUD apps

## ASGI (modern standard)

- Async-first — handles many concurrent connections
- Built-in WebSocket & HTTP/2 support
- Used by FastAPI, Starlette, Django 3+
- Essential for real-time & high-throughput apps

**How it fits together:**

Client (Browser) → Uvicorn (ASGI Server) → ASGI Protocol → FastAPI (Your App) → Response (JSON)

```
# Every ASGI app is an async callable
async def app(scope, receive, send):
    ... # FastAPI handles this for you
```

**Why should you care?**

You never write ASGI directly — but understanding the stack helps you debug deployments, choose the right server, and understand why async matters.

# Path Parameters

Extract values from the URL path

main.py

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

`{user_id} → user_id: int`

URL placeholder maps to function parameter. The type hint (int) validates automatically.

**Try these URLs:**

**GET /users/42**
`{"user_id": 42}` 200 OK

**GET /users/abc**
422 Validation Error 422

**GET /users/3.14**
422 Validation Error 422

Key Insight: The : int type hint is not just documentation — FastAPI uses it to validate and convert the parameter at runtime.

# Query Parameters

Filter and paginate with ?key=value

main.py

```
@app.get("/users")
def list_users(
    skip: int = 0,
    limit: int = 10,
    active: bool = True
):
    return db[skip : skip + limit]
```

**The Rule**

Parameters NOT in the URL path → query parameters.
Default values make them optional.

**URL examples:**

`/users`

Uses defaults: skip=0, limit=10

`/users?skip=20&limit=5`

Custom pagination

`/users?active=false`

Bool coercion: 'false' → False

`/users?limit=abc`

422 — 'abc' is not an integer

# Why type hints?

A modern Python web framework — fast to code, fast to run

### Type Hints → Validation

Write Python type hints, get automatic request validation, editor autocomplete, and error messages for free.

### Type Hints → Documentation

Your type hints generate interactive Swagger UI docs automatically. Code IS the documentation.

### Type Hints → Performance

Built on Starlette (ASGI) and Pydantic. As fast as Node.js/Go, with Python's readability.

### The Key Insight

In FastAPI, you write one thing:
Python type hints

And you get three things for free:
✓ Input validation
✓ API documentation
✓ Editor support

**Write hints once → get everything**

# Request Body with Pydantic

Validate complex JSON payloads with models

models + route

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    email: str
    age: int = 0 # optional, default 0

@app.post("/users")
def create_user(user: UserCreate):
    return {"name": user.name,
            "id": 1}
```

## What Pydantic does for you

- ✓ Parses JSON body automatically
- ✓ Validates all field types
- ✓ Applies default values
- ✓ Returns 422 with details on failure
- ✓ Generates JSON Schema for docs

**Example request:**

```
POST /users
{"name": "Alice",
 "email": "a@b.com",
 "age": 25}
```

# Pydantic Validation

Constrain fields with Field() and custom validators

Field() constraints

```
from pydantic import BaseModel, Field

class Product(BaseModel):
    name: str = Field(
        min_length=1,
        max_length=100
    )
    price: float = Field(gt=0)
    quantity: int = Field(ge=0, le=10000)
    sku: str = Field(
        pattern=r"^[A-Z]{3}-\d{3}$"
    )
```

Custom validator

```
from pydantic import field_validator

class Order(BaseModel):
    sku: str
    quantity: int

    @field_validator("sku")
    @classmethod
    def normalize_sku(cls, v):
        v = v.strip().upper()
        if " " in v:
            raise ValueError(
                "SKU cannot have spaces"
```

**422 Validation Error Response:**

{"detail": [{"loc": ["body", "price"], "msg": "Input should be greater than 0", "type": "greater_than"}]}

# Response Models

Control exactly what your API returns

Separate input/output models

```
class UserCreate(BaseModel):
    name: str
    email: str
    password: str    # input only!

class UserResponse(BaseModel):
    id: int
    name: str
    email: str       # no password!

@app.post("/users", response_model=UserResponse)
def create_user(user: UserCreate):
    db_user = save_to_db(user)
    return db_user    # password filtered out
```

## Why separate models?

**Security**
Never leak passwords or internal fields

**Documentation**
Swagger shows the exact response shape

**Consistency**
Every response matches the contract

**Pattern: Create → Response → DB**

# Status Codes & Error Handling

Communicate outcomes clearly with HTTP status codes

Error handling pattern

```
from fastapi import HTTPException, status

@app.post("/users",
    status_code=status.HTTP_201_CREATED)
def create_user(user: UserCreate):
    if db.email_exists(user.email):
        raise HTTPException(
            status_code=409,
            detail="Email already registered"
        )
    return db.create(user)

@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = db.get(user_id)
    if not user:
        raise HTTPException(
            status_code=404,
            detail=f"User {user_id} not found"
        )
    return user
```

## Status Code Cheat Sheet

| | |
|---|---|
| 200 | OK — successful GET/PUT |
| 201 | Created — successful POST |
| 204 | No Content — successful DELETE |
| 400 | Bad Request — client error |
| 404 | Not Found — resource missing |
| 409 | Conflict — duplicate resource |
| 422 | Validation Error — bad data |
| 500 | Server Error — your bug |

# Complete CRUD

All four operations on a resource

FullCRUD

```
# CREATE
@app.post("/items",status_code=201,response_model=ItemOut)
defcreate_item(item:ItemCreate):
    return db.insert(item)

# READ (list+single)
@app.get("/items")
deflist_items(skip:int=0,limit:int=20):
    return db.items[skip:skip+limit]

@app.get("/items/{item_id}",response_model=ItemOut)
defget_item(item_id:int):
    ifnot(item:=db.get(item_id)):
        raiseHTTPException(404,"Notfound")
    return item

# UPDATE
@app.put("/items/{item_id}",response_model=ItemOut)
defupdate_item(item_id:int,item:ItemCreate):
    return db.update(item_id,item)

# DELETE
@app.delete("/items/{item_id}",status_code=204)
defdelete_item(item_id:int):
    db.delete(item_id)
```

## HTTP → CRUD

**POST**
Create → 201

**GET**
Read → 200

**PUT**
Update → 200

**DELETE**
Delete → 204

Notice: every operation has the right status code

# Dependency Injection

Share logic across routes with Depends()

## Without DI — repeated everywhere

Repetitive

```
@app.get("/users")
def list_users(skip: int = 0, limit: int = 10):
    if limit > 100:        # repeated!
        limit = 100
    return users[skip:skip+limit]

@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    if limit > 100:        # copy-paste!
        limit = 100
    return items[skip:skip+limit]
```

## With DI — define once, inject

Reusable

```
from dataclasses import dataclass

@dataclass
class Pagination:
    skip: int = 0
    limit: int = 10
    def __post_init__(self):
        self.limit = min(self.limit, 100)

@app.get("/users")
def list_users(pg: Pagination = Depends()):
    return users[pg.skip : pg.skip+pg.limit]
```

**How Depends() works:**

1. FastAPI sees Depends() and calls your function/class before the route
2. The return value is injected as a parameter — here a Pagination object
3. Works with functions, generators (yield for cleanup), and classes
4. Dependencies can depend on other dependencies → build a chain

# JWT Authentication

Protect routes with token-based auth using Depends()

auth/deps.py

```
from jose import jwt
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

oauth2 = OAuth2PasswordBearer(tokenUrl="/login")
SECRET = "your-secret-key"

def get_current_user(
    token: str = Depends(oauth2)
):
    try:
        data = jwt.decode(token, SECRET, ["HS256"])
        return data["sub"]  # username
    except jwt.JWTError:
        raise HTTPException(401, "Invalid token")
```

Request flow: Bearer token → OAuth2PasswordBearer extracts it → get_current_user decodes → Route gets user

## Using the dependency:

Protected routes

```
# Any route can require auth
@app.get("/me")
def read_profile(
    user: str = Depends(get_current_user)
):
    return {"username": user}

@app.get("/admin")
def admin_panel(
    user: str = Depends(get_current_user)
):
    if user != "admin":
        raise HTTPException(403, "Forbidden")
    return {"msg": "Welcome, admin"}
```

Swagger UI adds a lock Authorize button automatically from OAuth2PasswordBearer!

# Organize with APIRouter

Split your API into modules as it grows

routers/users.py

```
# routers/users.py
from fastapi import APIRouter

router = APIRouter(
    prefix="/users",
    tags=["users"]
)

@router.get("/")
def list_users():
    return [...]

@router.post("/")
def create_user(user: UserCreate):
    return {...}
```

main.py

```
# main.py
from fastapi import FastAPI
from routers import users, items

app = FastAPI(title="My API")

app.include_router(users.router)
app.include_router(items.router)
```

Why? One file per domain. Easy to test, review, and navigate. New devs know where to add code.

**The result:**

```
GET  /users/   → list_users()
POST /users/   → create_user()
GET  /items/   → list_items()
POST /items/   → create_item()

SwaggerUI groups by tags automatically
```

# Project Structure

Where everything goes in a FastAPI project

File tree

```
my_api/
├── main.py        # App init + include routers
├── schemas.py     # Pydantic models (contracts)
├── dependencies.py  # Shared: DB, auth, etc.
├── database.py    # DB connection setup
│
├── routers/
│   ├── users.py   # /users/* endpoints
│   ├── items.py   # /items/* endpoints
│   └── auth.py    # /auth/* endpoints
│
├── models/
│   ├── user.py    # ORM / data models
│   └── item.py
│
├── tests/
│   ├── test_users.py
│   └── test_items.py
│
└── requirements.txt
```

## Each file has one job:

**main.py**
Entry point. Creates the app and wires routers together.

**schemas.py**
All Pydantic models. Your API's contract.

**dependencies.py**
Reusable Depends() functions for DB, auth.

**routers/**
One file per resource. Routes + business logic.

**models/**
Database models (SQLAlchemy, etc.)

**tests/**
Mirror routers/ structure. One test file per router.

# Testing Your API

Use TestClient to test routes without running a server

tests/test_users.py

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_create_user():
    response = client.post("/users", json={
        "name": "Alice",
        "email": "alice@example.com"
    })
    assert response.status_code == 201
    assert response.json()["name"] == "Alice"

def test_create_user_missing_name():
    response = client.post("/users", json={
        "email": "alice@example.com"
    })
    assert response.status_code == 422

def test_get_nonexistent_user():
    response = client.get("/users/9999")
    assert response.status_code == 404
```

## What to test:

- Happy path (valid input)
- Missing required fields (422)
- Resource not found (404)
- Duplicate resources (409)
- Wrong types in request (422)
- Response shape matches model

```
$ pytest tests/ -v
```

# Async & Await

Handle I/O-bound work without blocking

### Sync (blocking)

```
import requests

@app.get("/weather")
def get_weather():
    # Blocks the entire server
    r = requests.get(WEATHER_API)
    return r.json()
```

### Async (non-blocking)

```
import httpx

@app.get("/weather")
async def get_weather():
    # Other requests can proceed
    async with httpx.AsyncClient() as c:
        r = await c.get(WEATHER_API)
    return r.json()
```

**When to use async def:**

- ✓ External API calls (httpx)
- ✓ Async database queries (databases, asyncpg)
- ✓ File I/O with aiofiles
- ✓ Any await-able operation

**When to use regular def:**

- ✓ CPU-bound work (computation)
- ✓ Sync libraries (SQLAlchemy ORM)
- ✓ Simple operations with no I/O

FastAPI runs sync routes in a threadpool — they still work.

# Running with Uvicorn

FastAPI needs a server — Uvicorn is that server

FastAPI = the chef (knows how to cook) | Uvicorn = the restaurant (takes orders, serves food) | ASGI = the protocol between them

## Development

```
$ uvicorn main:app --reload

INFO: Uvicorn running on
 http://127.0.0.1:8000
INFO: Started reloader process
INFO: Waiting for changes...
```

## Production

```
$ uvicorn main:app \
   --host 0.0.0.0 \
   --port 8000 \
   --workers 4

# 4 worker processes for
# multi-core performance
```

Key flags: --reload (dev auto-restart) | --workers N (production parallelism) | --host 0.0.0.0 (expose to network) | --port 8000

# Automatic Documentation

Your code generates interactive docs — automatically

**/docs** **Swagger UI**

Interactive — try endpoints live in the browser. Send requests, see responses.

**/redoc** **ReDoc**

Beautiful read-only docs. Great for sharing with frontend teams and external consumers.

**/openapi.json** **OpenAPI Spec**

Machine-readable JSON. Use to generate client SDKs in any language.

Pro tip: Add descriptions to routes, tags for grouping, and examples in Pydantic models to make docs production-grade.

# Putting It All Together

A complete mini-API using everything we learned

schemas.py

```
# schemas.py
from pydantic import BaseModel, Field

class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    done: bool = False

class TaskOut(BaseModel):
    id: int
    title: str
    done: bool
```

**This uses everything:**

- ✓ Pydantic models (input + output)
- ✓ Field validation (min_length, max_length)
- ✓ Correct status codes (201, 404)
- ✓ response_model filtering
- ✓ HTTPException for errors

main.py

```
# main.py
from fastapi import FastAPI, HTTPException
from schemas import TaskCreate, TaskOut

app = FastAPI(title="Task API")
tasks = {}
counter = 0

@app.post("/tasks", status_code=201,
        response_model=TaskOut)
def create(task: TaskCreate):
    global counter; counter += 1
    tasks[counter] = {**task.dict(), "id": counter}
    return tasks[counter]

@app.get("/tasks/{id}", response_model=TaskOut)
def read(id: int):
    if id not in tasks:
        raise HTTPException(404, "Not found")
    return tasks[id]
```

# Common Pitfalls

Mistakes that trip up every FastAPI beginner

✗
```
return {"error": "not found"} # 200 OK
```

✓
```
raise HTTPException(404, "Not found")
```

Never return 200 with an error body — clients can't distinguish success from failure.

✗
```
@app.post("/users") # returns 200
```

✓
```
@app.post("/users", status_code=201)
```

POST creates a resource — always return 201 Created, not 200 OK.

✗
```
time.sleep(5) # in async route
```

✓
```
await asyncio.sleep(5)
```

Blocking calls in async routes freeze the event loop for ALL requests.

✗
```
def create(user: dict): # no validation
```

✓
```
def create(user: UserCreate): # validated
```

Raw dict bypasses all validation. Always use a Pydantic model.

Pro tip: Use mypy type checking and pytest to catch these issues before they reach production.

# Key Takeaways

1. **Start Simple**
   5 lines gets you a working API. Add complexity only when needed.

2. **Type Hints Drive Everything**
   Validation, docs, and editor support — all from type annotations.

3. **Pydantic = Your Contract**
   Separate input/output models. Validate at the boundary.

4. **Use Correct Status Codes**
   201 for creates, 204 for deletes, 404/422 for errors.

5. **Organize Early**
   APIRouter + project structure. One file per domain.

6. **Test Everything**
   TestClient + pytest. Happy path + error cases.

# Practice Exercise

Build a Bookstore API with these requirements:

1. Define BookCreate and BookOut Pydantic models with validation
2. POST /books — create a book (return 201)
3. GET /books — list all books with ?skip and ?limit
4. GET /books/{book_id} — get one book (404 if not found)
5. PUT /books/{book_id} — update a book
6. DELETE /books/{book_id} — delete (return 204)
7. Organize routes in routers/books.py
8. Write at least 3 tests with TestClient

fastapi.tiangolo.com | docs.pydantic.dev | Run: uvicorn main:app --reload